Automatic Generation of Formulae for Decidable Fragments of First-Order Logic

Manuele Borgia

June 26

Contents

Introduction

The automation of logical reasoning has been a central challenge in computer science since its earliest days. First-Order Logic (FOL), formalized in the foundations established by Frege's *Begriffsschrift* and later developed by Russell and Whitehead in *Principia Mathematica* [10], provides a language expressive enough to capture mathematical reasoning, while maintaining the structural precision required to admit computational treatment. Its applications span knowledge representation, formal verification, and artificial intelligence, where the ability to reason automatically over logical formulae has practical consequences ranging from software correctness to ontology reasoning.

While highly expressive, FOL is fundamentally constrained by the undecidability of its *satisfiability problem*, a theoretical limit proven by Church and Turing in the 1930s. This limitation has motivated two complementary lines of research. The first is the development of Automated Theorem Provers (ATPs) — systems such as Vampire [5], which employ saturation-based calculi and sophisticated heuristics to handle large classes of problems efficiently, operating within the theoretical bounds of semi-decidability. The second is the identification of *decidable fragments* of FOL: syntactically restricted subsets that retain sufficient expressive power for practical applications while admitting algorithms that always terminate with a correct answer.

This research analyzes four of the most studied decidable fragments: the Two-Variable Fragment (FO^2), the Guarded Fragment (GF), the Fluted Fragment (FF), and the Unary Negation Fragment (UNF). Each imposes distinct syntactic constraints — on the number of variables, the structure of quantification, or the use of negation — and each captures different classes of reasoning problems that arise naturally in areas such as description logics, database theory, and computational linguistics.

The practical evaluation of decision procedures for these fragments requires extensive testing on formulae that faithfully represent the syntactic diversity of each class. This need motivates the central contribution of this thesis: the design and implementation of **parametric random formula generators** for decidable fragments of FOL. Rather than relying on existing benchmark libraries, which are often sparse or biased toward specific problem structures, our generators produce formulae that are syntactically correct by construction, with configurable structural parameters controlling depth, predicate vocabulary, and quantifier density. A probabilistic *budget model* governs the generation pro-

cess, enabling systematic control over formula complexity. Generated formulae are validated using Vampire, ensuring that satisfiability labels are reliable. For the Two-Variable Fragment, a dedicated finite model construction provides an alternative, model-theoretic approach to generating satisfiable instances.

This thesis is organized as follows. Chapter 1 introduces the theoretical foundations of First-Order Logic. Chapter 2 presents the decidable fragments addressed by this work. Chapter 4 details the implementation and the probabilistic budget model of the formula generator. Finally, Chapter **??** discusses the validation process using Vampire.

Chapter 1

First-Order Logic

This chapter introduces the theoretical foundations on which this thesis is built. We present a concise overview of First-Order Logic (FOL), covering its syntax, semantics, and the fundamental undecidability of its satisfiability problem.

1.1 Syntax

A **signature** (or vocabulary) σ consists of a set of predicate symbols, each associated with a fixed *arity* $n \geq 0$, a set of function symbols, each with a fixed arity $n \geq 0$ (where arity-0 function symbols are called *constants*), and a countably infinite set of *variables* $\mathcal{V} = \{x_1, x_2, x_3, \ldots\}$.

Definition 1.1 (Terms)**.** *The set of* terms *over σ is defined inductively:*

- *every variable $x \in \mathcal{V}$ is a term;*
- *if f is a function symbol of arity n and $t_1, \ldots, t_n$ are terms, then $f(t_1, \ldots, t_n)$ is a term.*

A term containing no variables is called a ground term.

Definition 1.2 (Formulae)**.** *The set of* first-order formulae *over σ is defined inductively:*

- ***Atomic formulae.*** *If P is a predicate symbol of arity n and $t_1, \ldots, t_n$ are terms, then $P(t_1, \ldots, t_n)$ is an* atom*. The expression $t_1 = t_2$ is an* equality atom*.*
- ***Boolean connectives.*** *If φ and ψ are formulae, then so are $\neg\varphi$, $(\varphi \wedge \psi)$, $(\varphi \vee \psi)$, and $(\varphi \to \psi)$.*
- ***Quantifiers.*** *If φ is a formula and $x \in \mathcal{V}$, then $\exists x\, \varphi$ and $\forall x\, \varphi$ are formulae.*

An occurrence of a variable x in φ is *bound* if it lies within the scope of a quantifier $\exists x$ or $\forall x$; otherwise it is *free*. A formula with no free variables is called a *sentence*.

Definition 1.3 (Negation Normal Form)**.** *A formula is in* Negation Normal Form *(NNF) if negation $\neg$ is applied only to atomic formulae. Every formula can be converted to an equivalent formula in NNF by pushing negations inward using De Morgan's laws and the duality of quantifiers ($\neg\forall x\,\varphi \equiv \exists x\,\neg\varphi$ and $\neg\exists x\,\varphi \equiv \forall x\,\neg\varphi$).*

1.2 Semantics

Definition 1.4 (Structure)**.** *A σ-structure $\mathcal{M}$ consists of:*

- *a non-empty set $\Delta^{\mathcal{M}}$, called the* domain *(or* universe*);*
- *for each predicate symbol P of arity n, a relation $P^{\mathcal{M}} \subseteq (\Delta^{\mathcal{M}})^n$;*
- *for each function symbol f of arity n, a function $f^{\mathcal{M}} : (\Delta^{\mathcal{M}})^n \to \Delta^{\mathcal{M}}$.*

Definition 1.5 (Interpretation and Satisfaction)**.** *A* variable assignment *$\beta : \mathcal{V} \to \Delta^{\mathcal{M}}$ maps variables to domain elements. The* satisfaction relation *$\mathcal{M}, \beta \models \varphi$ is defined inductively:*

- *$\mathcal{M}, \beta \models P(t_1, \ldots, t_n)$ iff $(t_1^{\mathcal{M}}[\beta], \ldots, t_n^{\mathcal{M}}[\beta]) \in P^{\mathcal{M}}$;*
- *$\mathcal{M}, \beta \models t_1 = t_2$ iff $t_1^{\mathcal{M}}[\beta] = t_2^{\mathcal{M}}[\beta]$;*
- *$\mathcal{M}, \beta \models \neg\varphi$ iff $\mathcal{M}, \beta \not\models \varphi$;*
- *$\mathcal{M}, \beta \models \varphi \wedge \psi$ iff $\mathcal{M}, \beta \models \varphi$ and $\mathcal{M}, \beta \models \psi$;*
- *$\mathcal{M}, \beta \models \exists x\,\varphi$ iff there exists $d \in \Delta^{\mathcal{M}}$ such that $\mathcal{M}, \beta[x \mapsto d] \models \varphi$;*
- *$\mathcal{M}, \beta \models \forall x\,\varphi$ iff for all $d \in \Delta^{\mathcal{M}}$, $\mathcal{M}, \beta[x \mapsto d] \models \varphi$.*

For a sentence φ (no free variables), we write $\mathcal{M} \models \varphi$ and say $\mathcal{M}$ is a model *of φ.*

Definition 1.6 (Satisfiability and Validity)**.** *A formula φ is:*

- satisfiable *if there exists a structure $\mathcal{M}$ and assignment β such that $\mathcal{M}, \beta \models \varphi$;*
- valid *(or a* tautology*) if $\mathcal{M}, \beta \models \varphi$ for every structure $\mathcal{M}$ and every assignment β;*
- unsatisfiable *if it has no model.*

Note that φ is valid if and only if $\neg\varphi$ is unsatisfiable.

1.3 Decidability and Undecidability

Theorem 1.1 (Undecidability of FOL [2, 9])**.** *The satisfiability problem for first-order logic is undecidable: there is no algorithm that, given an arbitrary FOL sentence, determines in finite time whether it is satisfiable.*

This result, established independently by Church and Turing in the 1930s, fundamentally limits automated reasoning over full FOL. It has motivated the systematic study of syntactically restricted *fragments* of FOL whose satisfiability problem *is* decidable.

Chapter 2

Decidable Fragments

The fundamental undecidability of First-Order Logic necessitates the study of syntactically restricted *fragments* that achieve decidability while retaining enough expressivity for practical applications. This chapter introduces the decidable fragments targeted by our thesis.

Note on function symbols. The decidable fragments studied in this thesis are defined over a *purely relational* signature, i.e., a signature containing only predicate symbols, variables, and possibly constants. Function symbols of arity ≥ 1 are excluded, as they lead to undecidability. All logical analyses described in this thesis therefore operate within purely relational signatures.

2.1 The Two-Variable Fragment

The two-variable fragment of First-Order Logic, denoted by FO^2, is defined as the set of all FOL formulae that use at most two distinct variable names (typically x and y). Introduced in the context of the decision problem, FO^2 is one of the most prominent decidable fragments, balancing expressive power with a complexity class that remains within NExpTime [4].

2.1.1 Formal Syntax

The syntax of FO^2 is defined inductively. Let τ be a relational signature and $V = \{x, y\}$ be the set of available variables. The set of FO^2 formulae is defined as the smallest set such that:

Definition 2.1 (Two-Variable Formula)**.**

1. ***Atomic:***
   Every atomic formula $P(t_1, \ldots, t_n)$ using variables strictly from V is an FO^2 formula.

2. ***Boolean:***
   If ϕ and ψ are FO^2 formulae, then $\neg\phi$, $\phi \wedge \psi$, and $\phi \vee \psi$ are FO^2 formulae.

3. ***Quantification:***
   If ϕ is an FO^2 formula and $z \in \{x, y\}$, then $\exists z\phi$ and $\forall z\phi$ are FO^2 formulae.

2.1.2 Tree Representation and Variable Recycling

From a generative perspective, FO^2 formulae, like those in other fragments, are constructed via a top-down traversal of an Abstract Syntax Tree (AST). However, the generative constraint here differs significantly from the Fluted or Guarded fragments. Instead of maintaining a growing stack of variables (as in FF) or ensuring local variable coverage via guard atoms (as in GF), the AST traversal for FO^2 is globally bounded by the fixed variable pool $V = \{x, y\}$.

This imposes a *variable recycling* mechanism during the tree traversal. When constructing a deep AST branch, quantifiers must continuously reuse x and y, effectively "shadowing" or overwriting the variable bindings from higher up in the tree. For example, a valid FO^2 nested structure on the AST might look like:

$$\forall x\Big(P(x) \vee \exists y\big(R(x,y) \wedge \exists x(S(y,x))\big)\Big) \tag{2.1}$$

In this tree representation, the innermost existential quantifier over x rebinds the variable, severing the connection to the outermost universal quantifier. For the generator, this means that tracking the available variables is trivial (it is always a choice between x and y), but ensuring that the generated AST produces meaningful, non-trivial satisfiability challenges requires careful handling of this rebinding mechanism.

2.1.3 Finite Model Property for SAT Instances

2.1.4 Complexity

2.2 The Fluted Fragment

The landscape of decidable fragments is characterized by the trade-off between expressivity and complexity. By contrast to fragments defined by variable counting (like FO^2), the Fluted Fragment (FF) imposes a restriction on the ordering of variables. This variable-ordering discipline is significant because it preserves predicate arity and formula structure, focusing entirely on the logical application of predicates to variable sequences.

FF acts as a generalization of modal logic, sharing characteristics with the guarded fragments; however, it allows the negation of relation atoms, enabling the capture of enriched modal logics. Originally stemming from the work of Quine [7], the fragment's decidability is modernized and formalized by Pratt-Hartmann, Szwast, and Tendera [6].

2.2.1 Formal Syntax and the Suffix Property

The fundamental constraint of FF is the *suffix property*. An atomic formula is fluted if its arguments form a contiguous suffix of the variables currently in scope.

Definition 2.2 (Fluted Formula)**.** *The set of fluted formulae over X_i (where $0 \leq i \leq m$) is defined inductively:*

1. ***Atomic:***
   For any n-ary predicate $P \in \mathcal{P}$ such that $n \leq i$, the atomic formula $P(x_{i-n+1}, \ldots, x_i)$ is fluted over X_i.

2. ***Boolean:***
   If ϕ and ψ are fluted over X_i, then $\neg\phi$, $\phi \wedge \psi$, $\phi \vee \psi$, and $\phi \Rightarrow \psi$ are fluted over X_i.

3. ***Quantification:***
   If ϕ is fluted over X_{i+1}, then $\exists x_{i+1}\phi$ and $\forall x_{i+1}\phi$ are fluted over X_i.

This inductive structure entails that the quantification step expands the active variable scope, while atomic leaves are strictly bounded to the most recently introduced variables.

2.2.2 Tree Representation and the Scope Stack

From a generative perspective, the structure of a fluted formula is best understood through its Abstract Syntax Tree (AST). The variable-ordering discipline of FF naturally translates to a stack-based traversal during top-down generation.

Unlike FO^2, which recycles a fixed pool of variables, building an AST for the Fluted Fragment requires maintaining a dynamic "scope stack" of quantified variables that is passed down to child nodes. Each quantifier node conceptually pushes a new variable onto this stack.

Consider the following valid fluted formula:

$$\forall x_1 \forall x_2 \Big(P(x_1, x_2) \wedge \exists x_3 \big(R(x_2, x_3) \big) \Big) \tag{2.2}$$

Its syntactic structure can be visually represented as an AST, as shown in Figure 2.1.

When the AST traversal reaches a leaf node (an atomic formula), the suffix property acts as a strict generative constraint. For instance, when generating the atom R of arity 2 in Figure 2.1, the active stack is $[x_1, x_2, x_3]$. To comply with FF rules, the atom must exclusively consume the contiguous suffix (x_2, x_3). Attempting to construct an atom such as $Q(x_1, x_3)$ at that exact position would invalidate the formula. This inherent top-down predictability allows for an efficient, stack-driven generation strategy.

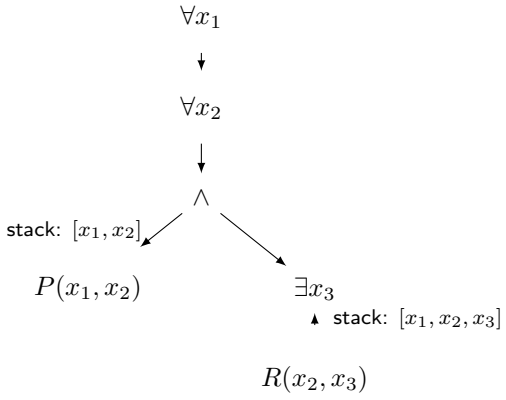

Figure 2.1: Syntactic tree of the fluted formula (2.2). The active variable stack is passed down the tree, and atomic leaves deterministically consume a contiguous suffix of it.

2.2.3 Complexity

The general Fluted Fragment exhibits non-elementary complexity, which scales heavily with the number of variables m [6]. This theoretical landscape poses a severe computational challenge, as attempting to prove unsatisfiability in FF can require exploring an enormously vast state space. In the context of this thesis, this justifies the necessity of our generation strategy: by algorithmically constructing structured FF formulae, we provide theorem provers with strictly compliant input to benchmark their limits against this non-elementary complexity.

2.3 The Guarded Fragment

The Guarded Fragment (GF), introduced by Andréka, van Benthem, and Németi [1], imposes structural constraints on the interaction between quantifiers and their sub-formulae. Quantification must be "guarded" by atomic formulae, which effectively restricts the domain of the quantifier to tuples satisfying a specific relational constraint.

2.3.1 Formal Syntax

The syntax of GF is defined inductively. Let τ be a relational signature. The set of guarded formulae is defined as the smallest set such that:

Definition 2.3 (Guarded Formula)**.** *1.* ***Atomic:***
Every atomic formula $P(t_1, \ldots, t_n)$ is a guarded formula.

2. ***Boolean:***
   If ϕ and ψ are guarded formulae, then $\neg\phi$ and $\phi \wedge \psi$ are guarded formulae.

3. ***Quantification:***
   Let $\phi(\bar{x}, \bar{y})$ be guarded and $G(\bar{x}, \bar{y})$ be an atomic guard containing all free variables of ϕ. Then $\exists\bar{y}(G(\bar{x}, \bar{y}) \wedge \phi)$ and $\forall\bar{y}(G(\bar{x}, \bar{y}) \Rightarrow \phi)$ are guarded formulae.

GF satisfies the *tree model property* [1], meaning every satisfiable guarded formula can be satisfied in a tree-structured model. This is the cornerstone of its decidability [3].

2.3.2 Tree Representation and Guard Generation

From a generative perspective, constructing a guarded formula requires a significantly different Abstract Syntax Tree (AST) traversal compared to the Fluted Fragment. While FF maintains a linear stack to enforce the suffix property, the generation of GF formulae relies on tracking a dynamic set of currently active free variables. In our top-down AST generation, whenever a quantifier over a set of new variables $\bar{y}$ is introduced, the fragment's rules dictate that a guard atom $G(\bar{x}, \bar{y})$ must be explicitly constructed, where $\bar{x}$ represents the variables currently in the active scope. To satisfy this, the generator must query the available vocabulary for a predicate whose arity exactly matches the combined size of the active scope and the newly bound variables.

Consequently, a quantified node in the AST is not a simple unary wrapper but a composite structure. For an existential quantification, the AST node branches into an $\wedge$ connective, with the specifically tailored guard atom G as its left child and the recursive sub-formula ϕ as its right child. The traversal then proceeds to generate the body of ϕ with the expanded active scope $\bar{x} \cup \bar{y}$. If no predicate of the required arity is available in the vocabulary to form a valid guard, the branch is deemed invalid, and the generator must backtrack by triggering a retry exception. This strict generative rule guarantees that the resulting AST inherently respects the localized, guarded variable dependencies that characterize the fragment.

2.3.3 Complexity

The satisfiability problem for the Guarded Fragment is ExpTime-complete [3]. This result highlights a significant increase in expressive power compared to basic propositional modal logic (which is typically PSpace-complete), while remaining much more tractable than the non-elementary complexity of the Fluted Fragment. Benchmarking provers against GF instances is particularly valuable for observing how solvers handle deeply nested local constraints without being overwhelmed by global variable dependencies.

2.4 The Unary Negation Fragment

The Unary Negation Fragment (UNFO), introduced by ten Cate and Segoufin [8], restricts the scope of negation rather than the quantifier pattern. While the Guarded Fragment restricts the placement of quantifiers to achieve a tree-like model, UNFO allows arbitrary quantification but isolates the primary source of undecidability in FOL: the unrestricted use of negation over multiple variables.

2.4.1 Formal Syntax

The syntax of UNFO is defined inductively. Let τ be a relational signature. The set of UNFO formulae is defined as the smallest set such that:

Definition 2.4 (Unary Negation Formula)**.** *1.* ***Atomic:***
Every atomic formula is an UNFO formula.

2. ***Boolean:***
If ϕ and ψ are UNFO formulae, then $\phi \wedge \psi$ and $\phi \vee \psi$ are UNFO formulae.

3. ***Negation:***
If ϕ is an UNFO formula such that $|free(\phi)| \leq 1$, then $\neg\phi$ is an UNFO formula.

4. ***Quantification:***
If ϕ is an UNFO formula, then $\exists x\phi$ and $\forall x\phi$ are UNFO formulae.

Decidability in UNFO is maintained by confining the complexity of negation to formulae with at most one free variable, effectively preventing the construction of undecidable contradictions in higher-arity predicates [8].

2.4.2 Tree Representation and Variable Dependencies

From a structural perspective, the syntax of an UNFO formula can be visualized through an Abstract Syntax Tree (AST), where the constraints of the fragment dictate the flow of variable dependencies. Unlike the Fluted Fragment (which relies on a strict variable stack) or the Guarded Fragment (which forces guard atoms at every quantification step), UNFO is highly permissive with positive connectives and quantifiers. The structural "bottleneck" occurs exclusively at the negation nodes.

Consider the following valid UNFO formula:

$$\forall x\Big(P(x) \vee \exists y\big(R(x,y) \wedge \neg Q(y)\big)\Big) \tag{2.3}$$

In this formula, the negation is applied solely to the sub-formula $Q(y)$, which contains exactly one free variable (y). This satisfies the unary restriction. Its AST representation is shown in Figure 2.2.

As illustrated in Figure 2.2, the positive branches (like the one leading to $R(x,y)$) can freely accumulate multiple variables in their scope. However, when

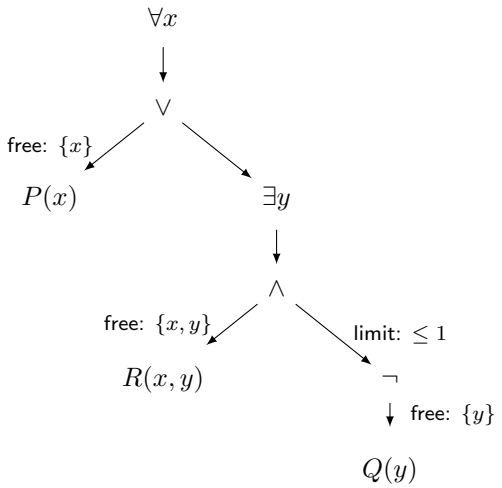

Figure 2.2: Syntactic tree of the valid UNFO formula (2.3). The negation node acts as a strict structural filter, ensuring that the sub-tree below it depends on at most one free variable.

the AST branches into a negation node, the set of free variables must be aggressively restricted.

Conversely, consider the following invalid formula:

$$\forall x \exists y \big(\neg R(x, y) \big) \tag{2.4}$$

This construction strictly violates the UNFO rules because the sub-formula under the negation, $R(x, y)$, contains two free variables (x and y). This creates a structural topology where branches underneath a negation node are inherently isolated in terms of variable dependencies. This architectural behavior perfectly mirrors the theoretical foundation of UNFO: it allows complex positive relations but forbids non-local negated structures, neutralizing the main trigger of undecidability in unrestricted FOL.

2.4.3 Complexity

The complexity of the satisfiability problem for UNFO is 2ExpTime-complete in the general case, but it drops to ExpTime-complete if the maximum arity of the predicates in the signature is fixed [8]. This makes UNFO an incredibly powerful theoretical tool: it offers a much larger expressive footprint than GF—allowing properties over arbitrarily many variables without guard atoms—while sitting within a comparable, well-understood complexity class.

2.5 Propositional Modal Logic and the Standard Translation

While the fragments discussed thus far (FO^2, FF, GF, UNFO) are defined purely within the vocabulary of First-Order Logic, their origins, properties, and constraints are deeply intertwined with Propositional Modal Logic (ML). Historically, modal logic was developed to reason about necessity and possibility, but from a modern computational perspective, it is best understood as a robustly decidable, tree-like logic for reasoning about relational structures, often evaluated over Kripke semantics.

The fundamental insight that connects ML to the decidable fragments of FOL is that modal operators can be viewed as "macros" for specific, heavily restricted patterns of first-order quantification. This connection establishes modal logic as the baseline or "minimal" decidable fragment, out of which fragments like GF and UNFO evolved as expressive generalizations.

2.5.1 The Standard Translation

The formal bridge between Propositional Modal Logic and First-Order Logic is the *Standard Translation* (ST). This translation maps any modal formula into an equivalent FOL formula over a relational signature containing only unary predicates (corresponding to modal propositions) and a single binary predicate R (corresponding to the accessibility relation between worlds).

Let $\mathcal{P}$ be a set of propositional variables. The standard translation $ST_x(\phi)$ of a modal formula ϕ evaluated at a world x is defined inductively:

Definition 2.5 (Standard Translation)**.**

1. ***Atomic:***
   For any propositional variable $p \in \mathcal{P}$, $ST_x(p) = P(x)$, where P is a unary predicate.

2. ***Boolean:***
   $ST_x(\neg\phi) = \neg ST_x(\phi)$ *and* $ST_x(\phi \wedge \psi) = ST_x(\phi) \wedge ST_x(\psi)$.

3. ***Existential Modality (Diamond):***
   $ST_x(\Diamond\phi) = \exists y\big(R(x,y) \wedge ST_y(\phi)\big)$.

4. ***Universal Modality (Box):***
   $ST_x(\Box\phi) = \forall y\big(R(x,y) \Rightarrow ST_y(\phi)\big)$.

As evident from the **Existential** and **Universal** rules, the translation of modal operators inherently produces guarded quantifiers. The binary relation $R(x,y)$ acts as a strict local guard, evaluating the sub-formula $ST_y(\phi)$ only in worlds y that are directly accessible from the current world x.

2.5.2 Tree Representation and Variable Alternation

A remarkable structural property of the Standard Translation is its extreme parsimony with variables. Although the translation of nested modal operators

(e.g., $\Diamond\Box\Diamond p$) conceptually moves through a sequence of distinct worlds, the resulting FOL formula does not require an unbounded number of variable names.

From an Abstract Syntax Tree (AST) perspective, building the translation requires tracking the current context node (the world x). When a modal operator expands the AST by introducing a quantifier over a new world, the target variable y only needs to be distinct from the immediately preceding world x. Once the traversal moves deeper into the tree, the previous variable x is no longer actively referenced in the local scope and can be safely overwritten.

Consequently, any modal formula can be translated using exactly two variables—conventionally denoted as w_1 and w_2—which strictly alternate as the depth of the modal nesting increases. For instance, the modal formula $\Diamond\Diamond p$ translates into:

$$\exists w_2\Big(R(w_1, w_2) \wedge \exists w_1\big(R(w_2, w_1) \wedge P(w_1)\big)\Big) \tag{2.5}$$

This variable alternation demonstrates that standard Modal Logic can be naturally embedded into FO^2, as it relies exclusively on a two-variable pool with recycling. Furthermore, because its quantifiers are always shielded by the accessibility relation R, it perfectly embeds into the Guarded Fragment (GF). Finally, because its properties are strictly unary ($P(x)$), it inherently satisfies the restrictions of the Unary Negation Fragment (UNFO).

2.5.3 Summary

In conclusion, Propositional Modal Logic serves as the central node connecting all the fragments studied in this chapter.

- The **Two-Variable Fragment** generalizes ML by abandoning the guard relation entirely, relying instead on the strict two-variable limit and recycling.
- The **Fluted Fragment** generalizes ML by viewing the sequence of modal transitions as a strict variable-ordering discipline (the scope stack). Crucially, unlike GF, FF allows the negation of relational atoms, enabling the embedding of enriched modal logics such as Boolean modal logic.
- The **Guarded Fragment** generalizes ML by maintaining the strict guard mechanism but extending it to relations of arbitrary arity.
- The **Unary Negation Fragment** generalizes ML by lifting restrictions on positive quantification entirely, demanding only that negation remains structurally simple (unary).

By understanding the standard translation, we obtain a clear topological map of the boundaries of decidability in First-Order Logic. Modal logic acts as the minimal intersection point, while FO^2, FF, GF, and UNFO represent different structural paths to extend its expressivity without crossing the threshold into undecidability. This provides the necessary theoretical foundation for the generation algorithms proposed in this thesis.

Chapter 3

System Architecture

This chapter ...

3.1 Lexical Abstraction

The foundational layer of the architecture handles the definition of the system's logical alphabet. Every syntactic element is encapsulated within the `Symbol` structure and strictly typed through the `SymbolType` enumeration. This system systematically defines all standard logical constructs:

punctuation ((,)), negation ($\neg$), connectives ($\wedge$, $\vee$, $\rightarrow$), quantifiers ($\forall$, $\exists$), equality ($=$), variables (x_1, x_2, ..), and predicates (A_1 for unary predicates, B_1 for binary predicates, ..).

To instantiate these entities, the creation of symbols is delegated to dedicated static **factory methods**, such as `Symbol::var()` for variables, or `Symbol::pred()` for predicates. The implementation of this creational pattern guarantees encapsulation and the enforcement of domain invariants right at the moment of creation.

The internal representation is built upon an Abstract Syntax Tree (AST). This hierarchical data structure intrinsically ensures that generated formulas are well-formed *by construction.*

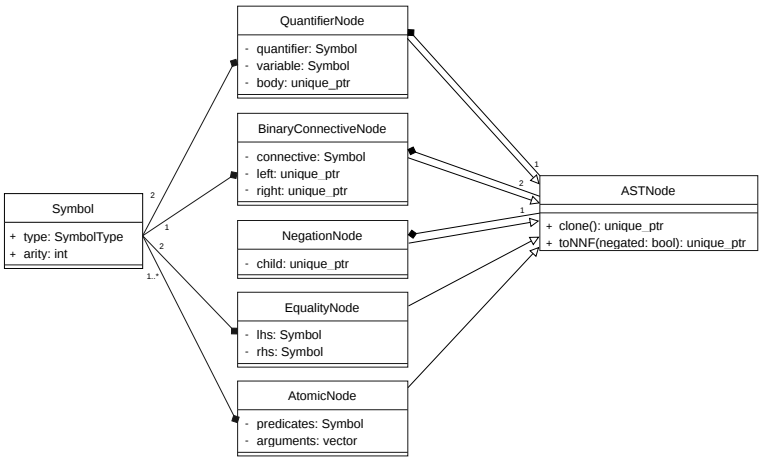

Figure 3.1: Class Diagram representing the AST architecture. The abstract base class `ASTNode` acts as the root, with specific final classes implementing the concrete nodes. Composition relationships highlight how nodes manage their children through exclusive ownership.

The tree's structural integrity relies on strict ownership and safe memory management. Complex nodes (such as `NegNode`, `BinaryConnNode`, and `QuantifierNode`) encapsulate their children via exclusive smart pointers (`std::unique_ptr`). This design perfectly mirrors the compositional nature of logical formulas: parents exclusively own their branches, guaranteeing automatic memory cleanup upon root destruction without the runtime overhead of a garbage collector.

This robust memory model synergizes with pure polymorphism to execute complex logical transformations, such as the conversion to **Negation Normal Form** (`toNNF()`). Relying on polymorphic recursion, each node independently applies type-specific rules to dynamically generate a structurally modified branch. For instance, an implication within a `BinaryConnNode` dynamically rewrites itself into an equivalent disjunction (`OR`) with a negated left child, seamlessly propagating the transformation down its exclusively owned subtrees.

3.2 Formula Generation

Once the logical alphabet and the structural foundation (AST) are defined, the system requires a robust engine to dynamically assemble these components into well-formed logical expressions. This responsibility is delegated to the `FormulaBuilder`, a core component designed specifically to orchestrate the generation of the Abstract Syntax Tree.

From an architectural standpoint, this component is driven by the **Template Method** design pattern to balance shared orchestration logic with strict

fragment-specific syntactic rules. The abstract base class `FormulaBuilder` establishes the invariant algorithmic skeleton: it handles the recursive descent mechanism, the retry loops, and the pre-processing configurations. However, it intentionally defers the instantiation of leaf nodes and the logic for satisfiability to its derived classes through pure virtual methods, specifically `buildAtomic()`, `generateSAT()`, and `buildComponentUNSAT()`.

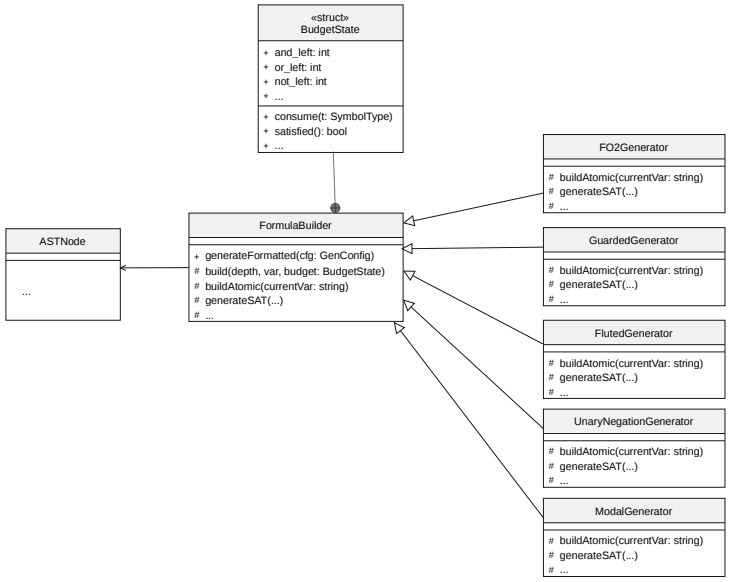

Figure 3.2: UML Class Diagram illustrating the Template Method pattern. `FormulaBuilder` defines the algorithmic skeleton, while concrete generator classes implement the abstract hooks to enforce their specific syntactic rules without duplicating the orchestration logic.

As illustrated in the class diagram (Figure 3.2), concrete implementations—such as the `FO2Generator`, `GuardedGenerator`, and `ModalGenerator`—act as specific strategies, injecting their domain-specific rules into these structural hooks. This polymorphic approach guarantees that each logical fragment can rigorously enforce its own restrictions without altering the central tree-building engine.

To manage the complexity of the recursive generation without heavily polluting method signatures, the architecture implements the **Context Object** pattern through a nested `BudgetState` structure. This state object encapsulates a mutable context for a single building session, dynamically tracking the remaining allowance for connectives and quantifiers. The state semantics are strictly defined: a value of -1 indicates an unconstrained resource, 0 completely forbids the usage of a specific symbol, and any value greater than 0 represents a limited resource that is meticulously decremented down the recursive call stack.

Because logic generation with exact structural constraints can be mathematically unpredictable, the system is engineered for high resilience. The main entry point, `generateFormatted()`, wraps the entire generation process in a retry loop protected by a try-catch mechanism. If a partially generated AST violates the strict limits defined in the `BudgetState`, the local scope throws a custom `BudgetRetryException`. The orchestrator immediately catches this exception, safely discards the invalid tree, and restarts the generation process up to a predefined threshold. This design guarantees that the final output either perfectly satisfies the configuration object constraints or gracefully terminates with an `std::invalid_argument` exception if the requested shape is fundamentally unsatisfiable.

3.3 Fragments Generation

3.4 The FO2 SAT Model and Visitor

To generate logical formulas that are strictly satisfiable (SAT) within the two-variable fragment (FO2), the system's architecture must transition from purely syntactic assembly to active semantic evaluation. This complex responsibility is orchestrated by the `FO2SATGenerator`, which extends the base `FO2Generator` to inject mathematical validation directly into the generation pipeline[cite: 10]. From a software design perspective, this process relies on the strict separation of concerns between the structural representation of the formula, the mathematical domain, and the evaluation algorithm.

The mathematical ground truth required for semantic validation is encapsulated within the `FiniteModel` class[cite: 11]. Operating as a Domain Model, this component dynamically generates and stores randomized boolean interpretations for both unary and binary predicates over a specified finite domain size[cite: 11]. By abstracting the domain logic, the model provides a clean, stateless interface (`evalAtom()`) to query the truth value of any given assignment without exposing its internal combinatorial matrices[cite: 11].

The most critical architectural challenge in this phase is traversing and evaluating the Abstract Syntax Tree without polluting the pure data structure of the nodes with domain-specific semantic logic. The architecture elegantly resolves this by implementing the **Visitor Pattern**[cite: 8]. The `FO2Evaluator` class acts as the concrete visitor, realizing the `ASTVisitor` interface to enable double-dispatch traversal across the tree[cite: 8]. As it visits specific node types—such as `AtomicNode`, `QuantifierNode`, or `BinaryConnNode`—the evaluator dynamically computes their boolean truth values against the current variable assignment and the referenced `FiniteModel`[cite: 8].

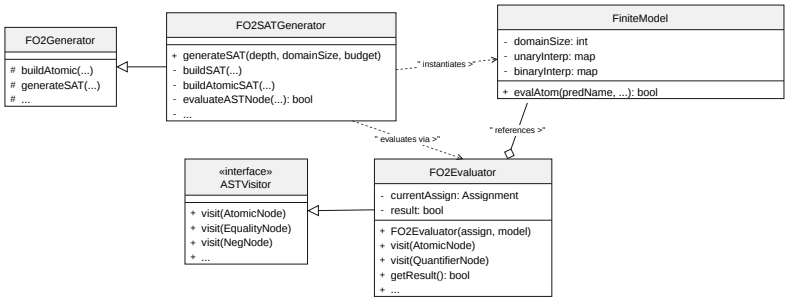

Figure 3.3: UML Class Diagram detailing the semantic SAT generation architecture. The `FO2SATGenerator` orchestrates the `FiniteModel`. It heavily relies on the **Visitor Pattern** via the `FO2Evaluator` to safely traverse the AST and compute semantic truth values without altering the underlying node structures.

As depicted in Figure 3.3, the `FO2SATGenerator` acts as the central coordinator of these patterns[cite: 10]. During the recursive construction of a formula, it acts as a client that proposes candidate sub-trees and immediately validates them by instantiating an `FO2Evaluator`[cite: 9, 10]. If a generated branch does not satisfy the semantic targets dictated by the `FiniteModel`, the generator leverages this immediate feedback to dynamically adjust the structure, such as wrapping the branch in a `NegNode` or falling back to a tautology, ensuring that the final output is structurally sound and mathematically satisfiable by construction[cite: 9].

3.5 Overview

Chapter 4

Implementation

The core contribution of this thesis is the design and implementation of a modular, parametric system for generating random formulae that belong to specific decidable fragments of First-Order Logic. Generating structured logical formulae is a non-trivial task; purely random syntactic string generation frequently results in ill-formed or trivially unsatisfiable expressions. To overcome this, our implementation relies on a highly configurable, probabilistic architecture centered around a robust internal data structure.

4.1 Abstract Syntax Tree (AST) Representation

4.2 The Probabilistic Budget Model

4.3 Fragment-Specific Constraints

Bibliography

[1] Hajnal Andréka, Johan van Benthem, and István Németi. Modal languages and bounded fragments of predicate logic. *Journal of Philosophical Logic*, 27(3):217–274, 1998.

[2] Alonzo Church. A note on the Entscheidungsproblem. *Journal of Symbolic Logic*, 1(1):40–41, 1936.

[3] Erich Grädel. On the restraining power of guards. *Journal of Symbolic Logic*, 64(4):1719–1742, 1999.

[4] Erich Grädel, Phokion G. Kolaitis, and Moshe Y. Vardi. On the decision problem for two-variable first-order logic. *Bulletin of Symbolic Logic*, 3(1):53–69, 1997.

[5] Laura Kovács and Andrei Voronkov. First-order theorem proving and Vampire. In *Proceedings of CAV*, volume 8044 of *LNCS*, pages 1–35, 2013.

[6] Ian Pratt-Hartmann, Wieslaw Szwast, and Lidia Tendera. Quine's fluted fragment revisited. In *Proceedings of LICS*, 2018.

[7] Willard V. Quine. Algebraic logic and predicate functors. *The Ways of Paradox and Other Essays*, 1969.

[8] Balder ten Cate and Luc Segoufin. Unary negation. *Logical Methods in Computer Science*, 9(3), 2013.

[9] Alan M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42:230–265, 1936.

[10] Alfred North Whitehead and Bertrand Russell. *Principia Mathematica*, volume 1–3. Cambridge University Press, Cambridge, 1910–1913.